

Instrukcja uruchamiania i sprawdzania działania backendu na Windows

1. Cel dokumentu

Ten dokument opisuje, jak uruchomić backend lokalnie na systemie Windows oraz jak krok po kroku sprawdzić, czy działa poprawnie. Instrukcja jest przeznaczona dla programistów, którzy dołączają do projektu i chcą szybko uruchomić środowisko developerskie na własnej maszynie.

2. Wymagania wstępne

Do uruchomienia projektu potrzebne są:

- Windows 10 lub nowszy,
- Python 3.9 lub nowszy,
- pip,
- PowerShell albo Command Prompt,
- dostęp do katalogu `backend/`.

Projekt działa lokalnie na bazie **SQLite**, więc nie trzeba osobno uruchamiać PostgreSQL.

3. Struktura robocza

Zakładamy, że projekt ma strukturę:

```
project-root/  
|-- backend/  
| |-- app/  
| |-- tests/  
| |-- requirements.txt  
| '-- run.py  
|-- frontend/  
'-- mobile/
```

Cała instrukcja dotyczy wyłącznie katalogu `backend/`.

4. Uruchomienie projektu krok po kroku na Windows

4.1. 1. Przejdź do katalogu backend

W PowerShell:

```
cd backend
```

4.2. 2. Utwórz środowisko wirtualne

```
python -m venv .venv
```

Jeżeli komenda python nie działa, spróbuj:

```
py -m venv .venv
```

4.3. 3. Aktywuj środowisko

W PowerShell:

```
.venv\Scripts\Activate.ps1
```

W Command Prompt:

```
.venv\Scripts\activate.bat
```

Po aktywacji terminal powinien pokazać prefiks (.venv).

4.4. 4. Zainstaluj zależności

```
pip install -r requirements.txt
```

4.5. 5. Utwórz plik .env

W katalogu backend/ utwórz plik .env i wpisz:

```
FLASK_ENV=development
DATABASE_URL=sqlite:///app.db
SECRET_KEY=change-me
JWT_SECRET_KEY=change-me-too
CORS_ORIGINS=http://localhost:3000
APP_TIMEZONE=Europe/Warsaw
MAIL_SENDER=noreply@example.com
MAIL_BACKEND=console
```

4.6. 6. Usuń starą bazę, jeśli chcesz wystartować od zera

W PowerShell:

```
Remove-Item app.db -ErrorAction SilentlyContinue
Remove-Item instance\app.db -ErrorAction SilentlyContinue
```

W Command Prompt:

```
del app.db
del instance\app.db
```

Ten krok nie jest obowiązkowy, ale warto go wykonać przy pierwszym uruchomieniu lub po większych zmianach w seedzie.

4.7. 7. Uruchom backend

```
python run.py
```

Jeżeli python nie działa, użyj:

```
py run.py
```

4.8. 8. Oczekiwany efekt

W terminalu powinien pojawić się komunikat podobny do:

```
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000
```

Oznacza to, że backend działa lokalnie na porcie 5000.

5. Co robi uruchomienie przez run.py

Aktualnie `run.py` działa w trybie developerskim i wykonuje następujące kroki:

- tworzy aplikację Flask,
- inicjalizuje bazę danych,
- wykonuje `db.create_all()`,
- uruchamia seed danych testowych,
- startuje lokalny serwer HTTP.

To oznacza, że po uruchomieniu projektu baza SQLite zostaje przygotowana automatycznie i nie trzeba ręcznie wykonywać migracji.

6. Konta testowe dostępne po seedzie

Po starcie backendu dostępne są przykładowe konta:

- admin: `admin@example.com` / `Admin123!`
- terapeuta: `anna@example.com` / `Password123!`
- terapeuta: `piotr@example.com` / `Password123!`
- pacjent: `jan@example.com` / `Password123!`
- pacjent: `ola@example.com` / `Password123!`

7. Jak sprawdzić, czy backend działa

7.1. 1. Healthcheck

Najprostszy test:

```
curl http://127.0.0.1:5000/api/v1/health
```

Oczekiwany wynik:

```
{
  "data": {
    "status": "ok"
  }
}
```

Jeżeli `curl` nie działa zgodnie z oczekiwaniem w PowerShell, możesz po prostu otworzyć w przeglądarce:

```
http://127.0.0.1:5000/api/v1/health
```

7.2. 2. Lista usług

```
curl http://127.0.0.1:5000/api/v1/services
```

Oczekiwany wynik: odpowiedź JSON zawierająca listę usług w polu `data`.

7.3. 3. Logowanie

```
curl -X POST http://127.0.0.1:5000/api/v1/auth/login ^  
-H "Content-Type: application/json" ^  
-d '{"email":"jan@example.com","password":"Password123!\\"}'
```

W PowerShell można też użyć:

```
Invoke-RestMethod -Method POST ^  
-Uri "http://127.0.0.1:5000/api/v1/auth/login" ^  
-ContentType "application/json" ^  
-Body '{"email":"jan@example.com","password":"Password123!\\"}'
```

Oczekiwany wynik:

- kod 200,
- `accessToken`,
- `refreshToken`,
- dane użytkownika.

7.4. 4. Endpoint wymagający autoryzacji

Po zalogowaniu należy skopiować `accessToken` i wykonać:

```
curl http://127.0.0.1:5000/api/v1/users/me ^  
-H "Authorization: Bearer <accessToken>"
```

W PowerShell:

```
Invoke-RestMethod -Method GET ^  
-Uri "http://127.0.0.1:5000/api/v1/users/me" ^  
-Headers @{ Authorization = "Bearer <accessToken>" }
```

Oczekiwany wynik: dane aktualnie zalogowanego użytkownika.

8. Przykładowy test przepływu pacjenta

Poniżej znajduje się najprostszy scenariusz manualny, który warto przejść po uruchomieniu projektu.

8.1. Krok 1. Zaloguj się

Wykonaj request do:

```
POST /api/v1/auth/login
```

i zapisz `accessToken`.

8.2. Krok 2. Pobierz listę usług

```
GET /api/v1/services
```

Z odpowiedzi zapisz `service_id`.

8.3. Krok 3. Pobierz terapeutów dla usługi

```
GET /api/v1/services/<service_id>/therapists
```

Z odpowiedzi zapisz `therapist_id`.

8.4. Krok 4. Pobierz dostępność

```
GET /api/v1/availability?serviceId=...&therapistId=...&from=...&to=...
```

Z odpowiedzi wybierz pierwszy poprawny `startAt`.

8.5. Krok 5. Zarezerwuj wizytę

```
curl -X POST http://127.0.0.1:5000/api/v1/appointments ^  
-H "Content-Type: application/json" ^  
-H "Authorization: Bearer <accessToken>" ^  
-d '{"serviceId": "<service_id>", "therapistId": "<therapist_id>", "startAt": "<slot_startAt>"}
```

Oczekiwany wynik:

- kod 201,
- status wizyty BOOKED.

8.6. Krok 6. Pobierz swoje wizyty

```
curl http://127.0.0.1:5000/api/v1/appointments/me ^  
-H "Authorization: Bearer <accessToken>"
```

Oczekiwany wynik: nowo utworzona wizyta znajduje się w liście.

8.7. Krok 7. Anuluj wizytę

```
curl -X POST http://127.0.0.1:5000/api/v1/appointments/<appointment_id>/cancel ^  
-H "Content-Type: application/json" ^  
-H "Authorization: Bearer <accessToken>" ^  
-d '{"reason": "Zmiana planu"}'
```

Oczekiwany wynik:

- kod 200,
- status CANCELLED_BY_PATIENT.

9. Jak uruchomić testy automatyczne na Windows

9.1. 1. Upewnij się, że jesteś w katalogu backend

```
cd backend
```

9.2. 2. Uruchom testy

Najbezpieczniej:

```
python -m pytest tests -q
```

Jeżeli python nie działa, użyj:

```
py -m pytest tests -q
```

Jeżeli środowisko wymaga jawnego ustawienia ścieżki modułów, użyj w PowerShell:

```
$env:PYTHONPATH="."
python -m pytest tests -q
```

albo w Command Prompt:

```
set PYTHONPATH=.
python -m pytest tests -q
```

9.3. 3. Co testują testy

Testy obejmują głównie:

- auth,
- katalog usług,
- dostępność,
- rezerwacje i anulowanie,
- review/admin/staff.

9.4. 4. Jak działa środowisko testowe

W testach:

- aplikacja uruchamiana jest w konfiguracji `testing`,
- baza jest tworzona od nowa,
- seed danych ładowany jest automatycznie.

10. Najczęstsze problemy i szybkie rozwiązania

10.1. Problem: ModuleNotFoundError: No module named 'app'

Uruchom testy tak:

```
python -m pytest tests -q
```

albo ustaw PYTHONPATH.

PowerShell:

```
$env:PYTHONPATH="."
python -m pytest tests -q
```

Command Prompt:

```
set PYTHONPATH=.
python -m pytest tests -q
```

10.2. Problem: backend działa, ale dane są nieaktualne

Usuń lokalną bazę SQLite i uruchom aplikację ponownie.

PowerShell:

```
Remove-Item app.db -ErrorAction SilentlyContinue
Remove-Item instance\app.db -ErrorAction SilentlyContinue
python run.py
```

Command Prompt:

```
del app.db
del instance\app.db
python run.py
```

10.3. Problem: test anulowania wizyty zwraca 403 zamiast 200

Najczęściej test wybrał slot, którego nie można już anulować zgodnie z regułą 24 godzin. W takim przypadku helper testowy powinien wybierać termin oddalony o więcej niż 24 godziny od bieżącego czasu.

10.4. Problem: różnice między datetime aware i naive

SQLite potrafi zwracać daty bez informacji o strefie. Jeżeli pojawia się błąd porównywania dat, należy ujednolicić obsługę czasu w serwisie lub dopiąć `timezone.utc` przed porównaniem.

10.5. Problem: PowerShell blokuje aktywację .ps1

Jeżeli przy aktywacji środowiska pojawia się błąd polityki wykonywania skryptów, uruchom PowerShell jako użytkownik i wykonaj:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Następnie ponownie aktywuj środowisko:

```
.venv\Scripts\Activate.ps1
```

11. Zalecany sposób pracy dla kolejnych programistów

Przy rozwijaniu backendu najlepiej pracować w tej kolejności:

1. doprecyzować model danych,
2. przygotować lub poprawić schemat wejścia/wyjścia,
3. zaimplementować logikę biznesową w serwisie,
4. wystawić endpoint w `app/api/`,
5. dodać testy,
6. zaktualizować seed, jeśli nowa funkcjonalność tego wymaga.

To pozwala utrzymać porządek i nie rozlewać logiki biznesowej po route'ach.

12. Krótka lista startowa dla nowego programisty

Jeśli wchodzisz do projektu po raz pierwszy, wykonaj:

1. przejdź do katalogu `backend/`,
2. utwórz `.venv`,
3. aktywuj środowisko,
4. zainstaluj zależności,
5. utwórz plik `.env`,
6. usuń starą bazę SQLite,
7. uruchom `python run.py`,
8. sprawdź `/api/v1/health`,
9. zaloguj się kontem z seeda,
10. przejdź podstawowy flow pacjenta,
11. uruchom `python -m pytest tests -q`.

Po tych krokach środowisko powinno działać poprawnie i być gotowe do dalszego rozwoju.